上海交通大学学士学位论文

JI 校企合作毕业设计项目的应用与数据库开发

姓　　名：杨雨昕　徐艺榕　李与时　罗旭腾　Jae-Heung Park

学　　号：520XXXXXXXX

导　　师：马澄斌

院　　系：机械与动力工程学院

学 科/专 业：工业工程

20XX 年 XX 月

A Dissertation Submitted to

Shanghai Jiao Tong University for the Degree of Bachelor

APP & DATABASE DESIGN AND DEVELOPMENT OF JI UNIVERSITY-INDUSTRY CAPSTONE PROJECT

Author: Yuxin Yang, Yirong Xu, Yushi Li, Xuteng Luo, Jae-Heung Pa

Supervisor: Ma Chengbin

School of XXXXXXX

Shanghai Jiao Tong University

Shanghai, P.R. China

June, 2024

上海交通大学
学位论文原创性声明

本人郑重声明：所呈交的学位论文，是本人在导师的指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的作品成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。本人完全知晓本声明的法律后果由本人承担。

学位论文作者签名：

日期：　　年　　月　　日

上海交通大学
学位论文使用授权书

本人同意学校保留并向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅。

本学位论文属于：

□**公开论文**

□**内部论文**，保密□ 1 年/□ 2 年/□ 3 年，过保密期后适用本授权书。

□**秘密论文**，保密____ 年（不超过 10 年），过保密期后适用本授权书。

□**机密论文**，保密____ 年（不超过 20 年），过保密期后适用本授权书。

（请在以上方框内选择打“✓”）

学位论文作者签名：　　指导教师签名：

日期：　　年　　月　　日　　日期：　　年　　月　　日

Abstract

Our project outlines a comprehensive solution to address the challenges of decentralized data management for capstone projects at JI. The primary goal of this project is to create a unified and scalable database system that centralizes project data, allowing for efficient retrieval and analysis.

Our design integrates various data types and incorporates advanced search capabilities to enhance the accessibility and usability of historical project data for both faculty and sponsors. We have chosen MySQL as our reliable and scalable relational database management system (RDBMS), complemented by a modular code architecture to support future expansion and maintenance.

We implement a custom OAuth2-based authentication system for its superior security and scalability, the Tanstack API request framework for its flexibility, performance, and security, the Algolia autocompletion framework for its ease of implementation and high performance, and object storage for its scalability, cost-effectiveness, and security.

The system is equipped with tailored authentication controls to ensure secure and personalized access for different user roles. Key features include powerful filtering mechanisms, responsive web interfaces, and an intuitive user experience designed to streamline interaction with the system. Our app is mainly tested on user satisfaction through surveys, efficiency through timing, and scalability through large test data sets, demonstrating that it works well.

Our project plan includes well-defined milestones and a systematic approach, encompassing technical analysis, prototype development, testing, and deployment. This structured methodology not only addresses current data management challenges but also establishes a strong foundation for future enhancements, aligning with the strategic objectives of JI's capstone program.

Key words: Capstone Project; Database; App Development; File Management

Contents

Contents ..... **II**

Chapter 1 Introduction **2**

Chapter 2 Design Specifications **3**

2.1 Customer Requirements 3

2.2 Customer Requirements Into Engineering Targets 4

2.3 Quality Function Deployment (QFD) 5

Chapter 3 Concept Generation **10**

3.1 Overview 10

3.2 Brainstorming 10

3.3 Morphological Chart 12

Chapter 4 Concept Selection Process **14**

4.1 Login System 14

4.1.1 Custom OAuth2-Based Authentication System (Score: 4.05) 14

4.1.2 MFA with Traditional Login (Score: 3.55) 15

4.1.3 SSO with Social and Enterprise Integration (Score: 3.55) 15

4.1.4 Conclusion 15

4.2 Search Engine 16

4.2.1 Tanstack API Request Framework (Total Score: 4.5) 16

4.2.2 PHPRTK API Request Framework (Total Score: 2.7) 17

4.2.3 API Request Framework (General) (Total Score: 4.0) 17

4.2.4 Angolia Autocompletion Framework (Total Score: 4.1) 17

4.2.5 ElasticSearch Autocompletion Framework (Total Score: 4.0) 18

4.2.6 General Autocompletion Framework (Total Score: 3.9) 18

4.2.7 Conclusion 18

4.3 Database Design 19

4.3.1 1. Object Storage (Total Score: 4.07) 19

4.3.2 2. File Storage (Total Score: 3.60) 20
4.3.3 3. Block Storage (Total Score: 3.27) 20
4.3.4 Conclusion 20

Chapter 5 Final Design 21
5.1 Overview of the Chosen Concept 21
5.1.1 System Architecture 21
5.2 Engineering Design Analysis 21
5.2.1 Specifications and Engineering Fundamentals 21
5.2.2 Approach for Determining Parameters 22
5.2.3 Engineering Decisions and Rationale 22
5.3 Detailed Layout Drawing 23
5.4 Components 24
5.5 Operation Description 25

Chapter 6 Manufacturing Plan 26
6.1 Materials and Manufacturing Processes 26
6.2 Programming and Simulation 26
6.3 Budget Considerations 27

Chapter 7 Validation Plan 28
7.1 Testing and Validation 28
7.2 Engineering Specifications 28
7.3 Testing Methods 28
7.4 Validation Process 28
7.5 Validation Results 29

Chapter 8 ECN 30

Chapter 9 Discussion 31
9.1 Critique of the Design 31
9.2 Strengths of the Design 31
9.3 Weaknesses of the Design 31
9.4 Improvements and Redesign Suggestions 32
9.5 Conclusion 33

9.6 Conclusion 34

Chapter 10 Future Work 35

10.1 Future Work 35

Acknowledgements 37

Reference 38

Appendixes 39

Abstract

Our project outlines a comprehensive solution to address the challenges of decentralized data management for capstone projects at JI. The primary goal of this project is to create a unified and scalable database system that centralizes project data, allowing for efficient retrieval and analysis.

Our design integrates various data types and incorporates advanced search capabilities to enhance the accessibility and usability of historical project data for both faculty and sponsors. We have chosen MySQL as our reliable and scalable relational database management system (RDBMS), complemented by a modular code architecture to support future expansion and maintenance.

We implement a custom OAuth2-based authentication system for its superior security and scalability, the Tanstack API request framework for its flexibility, performance, and security, the Algolia autocompletion framework for its ease of implementation and high performance, and object storage for its scalability, cost-effectiveness, and security.

The system is equipped with tailored authentication controls to ensure secure and personalized access for different user roles. Key features include powerful filtering mechanisms, responsive web interfaces, and an intuitive user experience designed to streamline interaction with the system. Our app is mainly tested on user satisfaction through surveys, efficiency through timing, and scalability through large test data sets, demonstrating that it works well.

Our project plan includes well-defined milestones and a systematic approach, encompassing technical analysis, prototype development, testing, and deployment. This structured methodology not only addresses current data management challenges but also establishes a strong foundation for future enhancements, aligning with the strategic objectives of JI's capstone program.

Key words: Capstone Project; Database; App Development; File Management

Chapter 1 Introduction

Over the past decade, JI has accumulated over 600 capstone projects, spread across various sources and formats. The photo below shows some pdf files of these projects. This decentralized storage has led to significant challenges in locating, accessing, and analyzing project data effectively. Faculty, students, and industry sponsors often struggle to find specific files, which hinders academic and research activities.

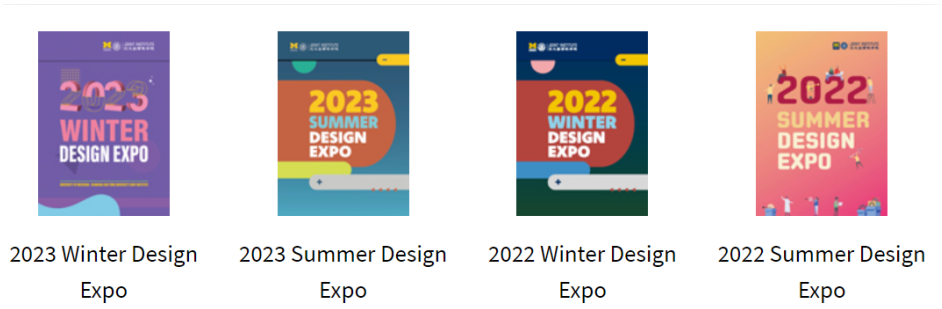

Figure 1.1 Example pdf files of capstone Projects [1]

To solve this problem, our project aims to develop a unified, scalable database system. This system will centralize all capstone project data, making it easily accessible through an intuitive interface. Our goal is to enable users to search for files using precise criteria such as project name, participant, or company name, thereby improving data management and usability.

By integrating all this data into a single, well-organized platform, we aim to enhance the efficiency of accessing and analyzing capstone projects. This project not only addresses immediate data management issues but also sets the stage for future enhancements and better collaboration between JI University and its industry partners.

Chapter 2 Design Specifications

Based on the information we provided in the introduction, we have derived the customer requirements out:

2.1 Customer Requirements

1. **Categorized Information and Powerful Filtering**:
   - Customers wanted precise information filtering through multiple criteria. To achieve this, we implemented a multi-criteria filtering system on the backend using precise SQL scripts for accurate data retrieval. This ensures users can quickly find the information they need, improving efficiency.
2. **System Stability**:
   - System stability is a key concern for customers. We implemented redundancy in critical components to prevent failures, using tools like Prometheus and New Relic to monitor system performance. We also ensured high code quality and provided comprehensive documentation for effective system maintenance.
3. **Simple and Intuitive Interface**:
   - Customers require a user-friendly interface. We used responsive frameworks like Bootstrap or Material-UI for optimal display, organizing data in cards or tables with sorting and pagination. Tooltips and modals enhance information accessibility without clutter.
4. **Future Expansion**:
   - The system needs to be highly scalable. We chose a scalable RDBMS like PostgreSQL, employing load balancing or sharding as needed. We designed a modular codebase to simplify updates and integrations, enhancing system adaptability and maintainability.
5. **Authentication and Account System**:
   - To ensure that users interact with interfaces and data based on their specific permissions, enhancing security and user experience, we implemented tailored access controls. This customization ensures that user access is aligned with their

specific roles and permissions.

6. **Integration with Existing Systems**:
   - Customers need integration with existing systems like Canvas. The system should be easily accessible from platforms like Canvas or other JI websites. To implement this, we need to contact the IT office to access source code on these websites and provide a link to our website.

2.2 Customer Requirements Into Engineering Targets

Our project is driven by several key customer requirements, each translated into specific engineering targets to ensure alignment with user needs. The following sections detail these requirements and their corresponding engineering targets.

1. **Categorized Information and Powerful Filtering**:
   - **Customer Requirement**: Customers need precise filtering of information through multiple criteria.
   - **Engineering Targets**:
     - Implement a multi-criteria filtering system with support for at least 10 distinct criteria.
     - Ensure that the filtering response time is less than 2 seconds for queries involving up to 1,000 records.
2. **System Stability**:
   - **Customer Requirement**: System stability and reliability are crucial for uninterrupted service.
   - **Engineering Targets**:
     - Achieve 99.9% uptime, with redundancy implemented in all critical system components.
     - Implement monitoring and alert systems using tools like Prometheus and New Relic to detect and resolve issues within 10 minutes.
3. **Simple and Intuitive Interface**:
   - **Customer Requirement**: The interface must be user-friendly and accessible.
   - **Engineering Targets**:
     - Use responsive frameworks such as Bootstrap or Material-UI to ensure com-

patibility with a range of devices and screen sizes.
  - Design the interface with sorting, pagination, and tooltips, ensuring that 90% of users can complete common tasks within 3 clicks.

4. **Future Expansion**:
   - **Customer Requirement**: The system must be scalable to handle future growth.
   - **Engineering Targets**:
     - Utilize PostgreSQL with scalability features such as load balancing or sharding to support a 50% increase in data volume annually.
     - Design the codebase to be modular, enabling feature updates and integrations with less than 5
5. **Authentication and Account System**:
   - **Customer Requirement**: Secure and role-based access controls are needed.
   - **Engineering Targets**:
     - Implement role-based access controls (RBAC) with at least 5 distinct user roles and permissions.
     - Ensure authentication processes comply with industry standards, achieving a 99.5% success rate in secure logins.
6. **Integration with Existing Systems**:
   - **Customer Requirement**: The system should integrate seamlessly with existing platforms like Canvas.
   - **Engineering Targets**:
     - Develop integration APIs to interface with Canvas and other JI systems, with an implementation time of less than 2 months.
     - Ensure that the integration provides single sign-on (SSO) functionality, reducing user login time by 30%.

2.3 Quality Function Deployment (QFD)

The Quality Function Deployment (QFD) process was employed to translate these customer requirements into actionable engineering targets. The QFD process involves mapping customer needs to technical requirements and prioritizing them based on importance and feasibility.

To develop the QFD, we began by identifying and categorizing customer requirements through surveys and feedback sessions. We then created a QFD matrix, where customer needs were aligned with specific engineering features. This matrix helped us prioritize engineering targets based on their impact on meeting customer requirements and their technical feasibility.

The QFD matrix revealed that categorized information and powerful filtering, along with system stability, were top priorities for customers. Consequently, these areas received the highest emphasis in our engineering targets, including stringent performance metrics and redundancy measures.

In summary, the QFD process ensured that our engineering targets are closely aligned with customer needs, providing a structured approach to achieving high customer satisfaction and system performance.

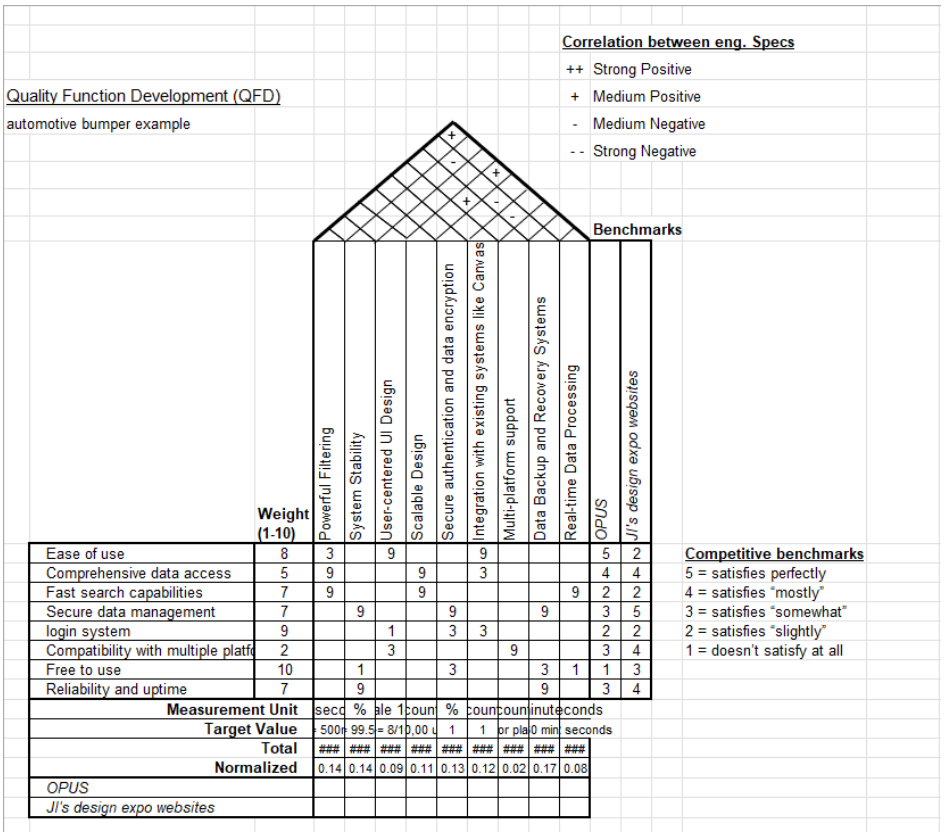

Figure 2.1 QFD

1. **Powerful Filtering**:
   - **Purpose**: To enable users to quickly navigate and locate specific data within the system.
   - **QFD Role**: The QFD process determines how critical powerful filtering is by evaluating its impact on customer satisfaction related to fast search capabilities and comprehensive data access. It helps prioritize development resources to enhance filtering features based on customer feedback.
   - **Unit**: Filter response time measured in milliseconds (ms).
2. **System Stability**:
   - **Purpose**: To enable users to quickly navigate and locate specific data within the system.
   - **QFD Role**: QFD assesses the impact of system stability on overall reliability and security. It translates the need for high uptime into specific technical requirements like redundant systems or robust error-handling mechanisms.
   - **Unit**: System uptime measured as a percentage of operational time over a given period (%).
3. **User-centered UI Design**:
   - **Purpose**: To enable users to quickly navigate and locate specific data within the system.
   - **QFD Role**: The process quantifies how much user satisfaction can be improved through UI design enhancements and helps align the design process with user preferences and usability standards.
   - **Unit**: User satisfaction index measured on a scale from 1 to 10, where 10 is the highest satisfaction.
4. **Scalable Design**:
   - **Purpose**: To enable users to quickly navigate and locate specific data within the system.
   - **QFD Role**: QFD helps determine the necessary architecture (like cloud-based services or server capabilities) to ensure scalability, influencing decisions about infrastructure investments and design approaches.
   - **Unit**: Maximum load capacity measured as the number of simultaneous users

the system can handle (users).

5. **Secure Authentication and Data Encryption**:
   - **Purpose**: To enable users to quickly navigate and locate specific data within the system.
   - **QFD Role**: It maps out how different security technologies and protocols can be integrated into the product, ensuring compliance with legal and ethical standards.
   - **Unit**: Compliance rate measured by the percentage of implemented security controls that meet industry standards (%).
6. **Integration with Existing Systems like Canvas**:
   - **Purpose**: To enable users to quickly navigate and locate specific data within the system.
   - **QFD Role**: QFD evaluates the technical and practical feasibility of such integrations, facilitating decision-making on API development and system compatibility.
   - **Unit**: Integration efficiency rated by the number of external systems seamlessly integrated (systems).
7. **Multi-platform Support**:
   - **Purpose**: To enable users to quickly navigate and locate specific data within the system.
   - **QFD Role**: QFD identifies the technical challenges and market demands for multi-platform compatibility, guiding the development of cross-platform solutions.
   - **Unit**: Compatibility score measured by the number of different platforms (operating systems, devices) successfully supported (platforms).
8. **Data Backup and Recovery Systems**:
   - **Purpose**: To enable users to quickly navigate and locate specific data within the system.
   - **QFD Role**: It determines the required frequency and methods of data backup, translating these into specific technical measures and infrastructure requirements.
   - **Unit**: Recovery time objective (RTO) measured in minutes, indicating the maximum allowable downtime in disaster recovery (minutes).

9. **Real-time Data Processing**:
    - **Purpose**: To enable users to quickly navigate and locate specific data within the system.
    - **QFD Role**: QFD facilitates the specification of necessary processing power and algorithms needed to achieve real-time performance, integrating customer needs with technical possibilities.
    - **Unit**: Processing latency measured in seconds, indicating the time taken from data input to actionable output (seconds).

Chapter 3 Concept Generation

3.1 Overview

The concept generation phase for our capstone design project involved extensive brainstorming and the use of morphological charts to explore different solutions for our system components. The primary goal was to identify and evaluate various options for the following categories: Login System, Database, Autocompletion, API Request Framework, Authentication, File Storage, and Search Engine. Here, we describe the major categories of concepts generated and the methods used.

3.2 Brainstorming

After conducting team brainstorming sessions to generate a wide range of ideas without immediate judgment or filtering. We encouraged open discussion to explore unconventional solutions and diverse perspectives. Finally we recorded all ideas for later evaluation and categorization.

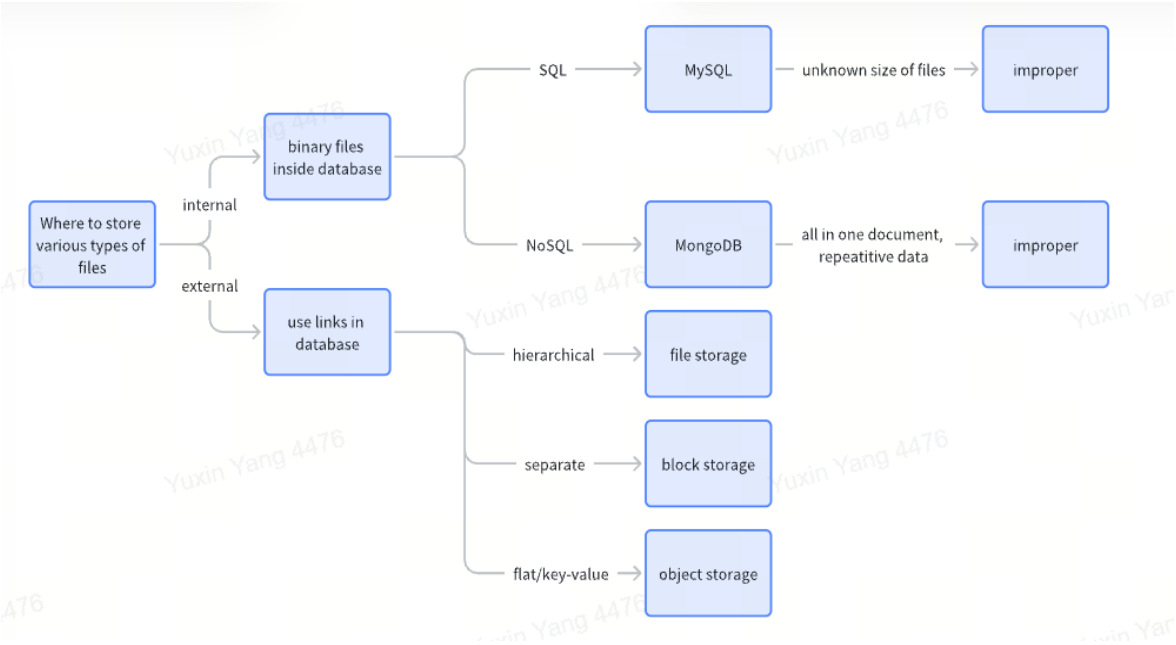

Figure 3.1 Ideas of database design

Component	Option 1	Option 2	Option 3	Option 4	Option 5
Authentication Method	Custom OAuth2-Based Authentication System	Multi-Factor Authentication (MFA)	Single Sign-On (SSO)	Biometric Authentication	Magic Link Authentication
User Interface	Custom Login Page	Social Login Buttons	Passwordless Login	Two-Step Verification	Captcha Integration
Security Measures	Encrypted Tokens	SMS/Email Verification	Time-Based One-Time Password (TOTP)	Hardware Security Keys	AI-Based Anomaly Detection
User Management	Role-Based Access Control	Self-Service Password Recovery	Centralized User Directory	Account Lockout Mechanisms	User Activity Monitoring

Figure 3.2 Ideas of login system design

Problem	Solution	Details
Difficult Access	Intuitive User Interface	**Technology**: React.js, Angular User-friendly design with a responsive interface. Implementing advanced search functionality using Tanstack Query for API requests.
Security	Custom OAuth2-Based Authentication System	**Technology**: OAuth 2.0, OpenID Connect Secure access for different user groups using a custom OAuth2-based system. Role-based permissions managed through a user roles database.
Organization	Metadata	**Technology**: Elasticsearch, Apache Solr Enhanced searchability and organization through metadata tagging and indexing. Use of search engine technology for efficient data retrieval.
Integration	Integration with Existing Systems	**Technology**: RESTful APIs, GraphQL Seamless integration with existing JI systems using APIs. Data synchronization with existing tools and platforms.
Collaboration	Collaboration Tools	**Technology**: Google Workspace, Microsoft Teams Tools to facilitate communication and teamwork, including shared documents and real-time collaboration.

Problem	Solution	Details
Data Analysis	Analytics and Reporting	**Technology**: Power BI, Tableau Analytical tools for data analysis and visualization. Generate reports and dashboards for academic and research purposes.
Data Security	Encryption	**Technology**: SSL/TLS, AES-256 Implementation of encryption for data storage and transmission to protect sensitive information.
Data Migration	Migration Plan	**Technology**: ETL Tools (Apache Nifi, Talend) Plan for migrating existing data using ETL (Extract, Transform, Load) processes. Data cleaning and standardization to ensure consistency and accuracy.

Table 3.1 Detailed technical solutions brainstorming.

3.3 Morphological Chart

We have decomposed the whole design into 3 sub-functions: login system, database design and search engine.

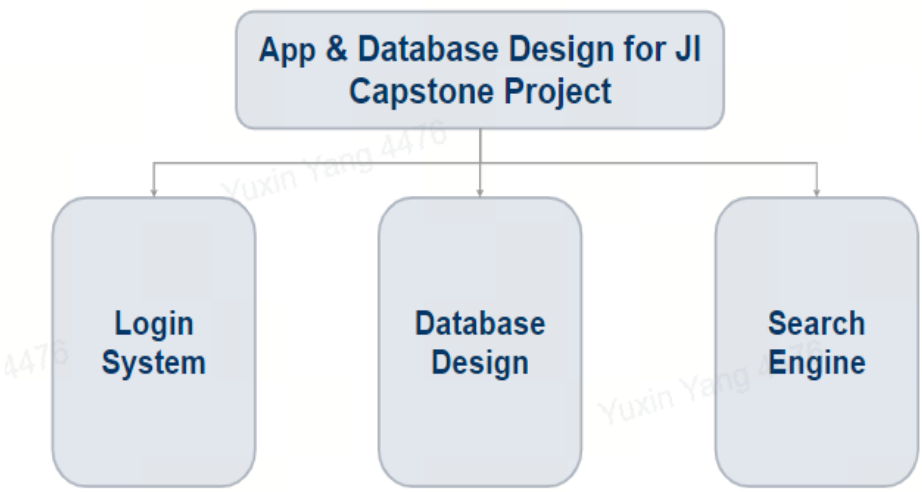

Figure 3.3 Decompose into sub-functions

- **Login System**: This sub-function handles user authentication and authorization. It ensures that users can securely log in and access the appropriate features based on their roles. Key considerations include security, ease of use, and scalability.
- **Database Design**: This sub-function focuses on the organization, storage, and management of data. It involves designing a robust schema that supports efficient data

retrieval and ensures data integrity.

- **Search Engine**: This sub-function provides advanced search capabilities to help users quickly locate relevant data. It integrates metadata tagging and indexing to enhance searchability and efficiency.

Based on these sub-functions, we make the morphological chart

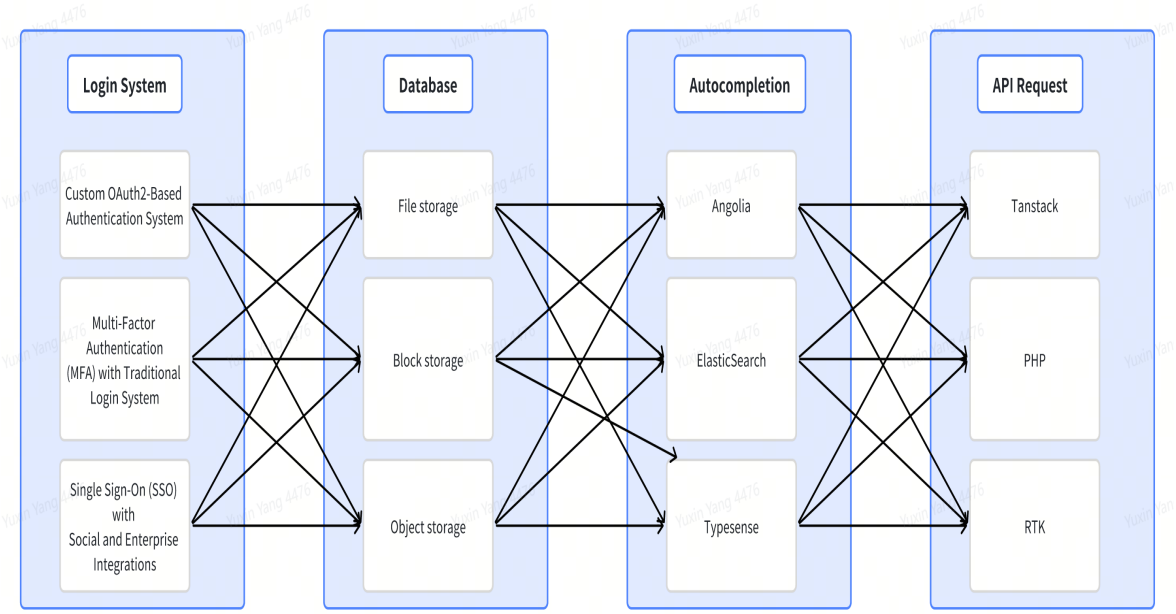

Figure 3.4 Morphological chart

Chapter 4 Concept Selection Process

To determine the best solution for the login system, we employed a concept selection process using a scoring matrix. This approach allows us to evaluate each concept against a set of criteria and make an objective decision based on quantitative data. The primary criteria used for evaluation are: Ease of Implementation, Security, Scalability, User Experience, Cost and Flexibility. Each criterion was assigned a weight reflecting its importance, and the concepts were scored accordingly.

4.1 Login System

For login system, we compare 3 possible solutions in the following decision matrix.

Critera	Weight	1. Custom OAuth2-Based Authentication	2. MFA with Traditional Login	3. SSO with Social and Enterprise Integration
Ease of Implementation	0.20	4	3	2
Security	0.25	5	5	4
Scalability	0.15	4	3	5
User Experience	0.15	4	3	5
Total Score	1.00	4.05(**Best!**)	3.55	3.55

Figure 4.1 Decision matrix for login system

4.1.1 Custom OAuth2-Based Authentication System (Score: 4.05)

Advantages:

- **High Security:** Provides robust security features through OAuth 2.0 and OpenID Connect, ensuring secure user authentication and authorization.
- **Flexibility:** Can be customized to fit various needs and integrate with other systems.
- **Scalability:** Scales well with increasing numbers of users and data.

Disadvantages:

- **Complex Implementation:** Requires careful implementation and management, which can be resource-intensive.
- **Requires Expertise:** Needs specialized knowledge to set up and maintain effectively.

4.1.2 MFA with Traditional Login (Score: 3.55)

Advantages:

- **Enhanced Security:** Adds an additional layer of security beyond traditional login methods.
- **Widely Understood:** Familiar to many users and administrators, reducing the learning curve.

Disadvantages:

- **Moderate Scalability:** May face challenges with scalability compared to more modern systems.
- **User Experience:** May be perceived as less user-friendly due to additional authentication steps.

4.1.3 SSO with Social and Enterprise Integration (Score: 3.55)

Advantages:

- **Excellent User Experience:** Simplifies the login process by allowing users to authenticate using existing social or enterprise accounts.
- **High Scalability:** Easily scales with large numbers of users and integrates with various platforms.

Disadvantages:

- **Security Concerns:** Reliance on third-party authentication providers can pose security risks.
- **Complex Integration:** Integrating with multiple social and enterprise systems can be complex and challenging.

4.1.4 Conclusion

After evaluating the concepts using the scoring matrix, the **Custom OAuth2-Based Authentication System** emerged as the best choice with the highest score of 4.05. This solution offers superior security and scalability, aligns well with customer specifications and

engineering requirements, and provides a flexible and robust framework for managing user authentication.

While the other concepts offer valuable features and advantages, they fall short in certain areas compared to the Custom OAuth2-Based Authentication System. Specifically, MFA with Traditional Login and SSO with Social and Enterprise Integration have their own strengths, but they do not match the overall effectiveness of the Custom OAuth2-Based Authentication System in terms of security, scalability, and ease of implementation.

4.2 Search Engine

For search engine, we compare 3 possible solutions for API request and 3 possible solutions for auto-completion.

Decision Criterion	Weight Vector	API Request Framework			Autocompletion Framework		
		Tanstack	PHP	RTK	**Angolia**	ElasticSearch	TypeSense
Ease of Implementation	0.2	4	3	2	5	3	4
Scalability	0.1	4	2	4	5	5	3
Performance	0.2	4	3	4	5	5	4
Cost	0.1	3	5	3	3	3	4
Security	0.2	4	3	4	4	5	4
Flexibility	0.2	4	2	4	4	5	4
Total Score	1	4.5	2.7	4.1	4.0	3.9	3.5

Figure 4.2 Decision matrix for search engine

4.2.1 Tanstack API Request Framework (Total Score: 4.5)

Advantages:

- **High Flexibility:** Provides excellent adaptability to various use cases and integrates well with other systems.
- **Good Performance:** Efficient in terms of speed and resource usage.
- **Secure:** Offers robust security features to protect against vulnerabilities.

Disadvantages:

- **Moderate Ease of Implementation:** While it has good documentation, setup can be complex compared to simpler frameworks.
- **Cost:** Comparable to other frameworks but does not offer significant cost benefits.

4.2.2 PHPRTK API Request Framework (Total Score: 2.7)

Advantages:

- **Simple Setup:** Generally easier to set up and integrate, with straightforward documentation.
- **Lower Cost:** Typically less expensive than more feature-rich frameworks.

Disadvantages:

- **Limited Flexibility:** Less adaptable to complex use cases and integrations.
- **Moderate Performance:** May not handle high loads as efficiently as other frameworks.
- **Security Concerns:** Lower security features compared to more advanced frameworks.

4.2.3 API Request Framework (General) (Total Score: 4.0)

Advantages:

- **Good Scalability:** Effective at handling increasing data loads.
- **Decent Performance:** Provides good speed and efficiency for API requests.
- **Reasonable Cost:** Balanced cost relative to features and benefits.

Disadvantages:

- **Moderate Ease of Implementation:** May require some technical expertise to integrate effectively.
- **Security:** May not have the highest level of security compared to specialized frameworks.

4.2.4 Angolia Autocompletion Framework (Total Score: 4.1)

Advantages:

- **Excellent Ease of Implementation:** User-friendly and straightforward to integrate.
- **High Performance:** Efficient and responsive in providing autocomplete suggestions.
- **Good Flexibility:** Adaptable to various use cases and integration scenarios.

Disadvantages:

- **Higher Cost:** Potentially more expensive, depending on scale and usage.
- **Security:** Slightly lower security rating compared to other options.

4.2.5 ElasticSearch Autocompletion Framework (Total Score: 4.0)

Advantages:

- **Scalability:** Excellent at handling large volumes of data and queries.
- **Strong Security:** Robust security features to protect data.
- **Good Performance:** Handles queries efficiently and quickly.

Disadvantages:

- **Moderate Ease of Implementation:** Integration can be complex and may require more technical expertise.
- **Higher Cost:** Can become expensive, especially at larger scales.

4.2.6 General Autocompletion Framework (Total Score: 3.9)

Advantages:

- **Decent Performance:** Provides acceptable speed and efficiency for autocomplete functionality.
- **Reasonable Flexibility:** Offers good adaptability to various use cases.

Disadvantages:

- **Moderate Security:** May not offer the highest security compared to specialized frameworks.
- **Cost:** May have a higher total cost depending on features and usage.

4.2.7 Conclusion

After evaluating the frameworks using the decision matrix, the **Tanstack API Request Framework** emerged as the best choice with the highest score of 4.5. This framework excels in flexibility, performance, and security, aligning well with our project' s requirements. It offers a robust and adaptable solution that integrates effectively with other systems while maintaining high performance and security standards.

For autocompletion frameworks, the **Angolia Autocompletion Framework** is the top choice with a score of 4.1, praised for its ease of implementation and high performance. Although the ElasticSearch Autocompletion Framework also performs well, it is slightly more

complex to integrate and more costly.

4.3 Database Design

For search engine, we compare 3 possible solutions for database design.

Decision Criterion	Weight Vector	File Storage		Block Storage		Object Storage	
		Score	Rating	Score	Rating	Score	Rating
Ease of Implementation	0.20	5	1.00	2	0.40	3	0.60
Scalability	0.07	2	0.13	4	0.27	5	0.33
Performance	0.13	3	0.40	5	0.67	4	0.53
Cost	0.20	4	0.80	2	0.40	3	0.60
Security	0.33	3	1.00	4	1.33	5	1.67
Flexibility	0.07	4	0.27	3	0.20	5	0.33
Total Score	1.00		3.60		3.27		4.07

Figure 4.3 Decision matrix for database design

4.3.1 1. Object Storage (Total Score: 4.07)

Advantages:

- **Ease of Implementation:** Provides a straightforward setup with well-documented APIs and tools.
- **Scalability:** Highly scalable, designed to handle vast amounts of data efficiently.
- **Cost:** Generally cost-effective for large-scale data storage.
- **Security:** Strong security features with built-in redundancy and encryption.
- **Flexibility:** Supports a wide variety of data types and access patterns.

Disadvantages:

- **Performance:** Performance can vary based on implementation and access patterns, though generally sufficient for most needs.
- **Complexity of Management:** May require more complex management and configuration compared to simpler storage solutions.

4.3.2 2. File Storage (Total Score: 3.60)

Advantages:

- **Ease of Implementation:** Typically easy to set up and integrate with existing systems.
- **Cost:** Often less expensive for smaller-scale applications.
- **Flexibility:** Supports various types of file data.

Disadvantages:

- **Scalability:** Less scalable compared to object storage; may require additional infrastructure as data grows.
- **Security:** Security features might be less robust compared to more modern solutions.
- **Performance:** Performance can degrade with increasing data volume.

4.3.3 3. Block Storage (Total Score: 3.27)

Advantages:

- **Performance:** Generally offers good performance for data-intensive applications.
- **Scalability:** Can be scaled, though it might require additional setup.

Disadvantages:

- **Ease of Implementation:** Setup and integration can be complex and require specific expertise.
- **Cost:** Can be more expensive, especially for large-scale storage needs.
- **Flexibility:** Less flexible in handling different types of data compared to object storage.

4.3.4 Conclusion

After evaluating the file storage solutions using the decision matrix, **Object Storage** emerged as the best choice with the highest total score of 4.07. It offers a robust combination of scalability, cost-effectiveness, and security, making it well-suited for handling large volumes of data efficiently.

While File Storage and Block Storage also provide valuable features, they fall short in certain areas compared to Object Storage. File Storage, though easier and cheaper for smaller scales, lacks scalability and robust security features. Block Storage, while performance is costlier and more complex to implement.

Chapter 5 Final Design

5.1 Overview of the Chosen Concept

The chosen concept integrates the following components:

- **Login System**: Custom OAuth2-Based Authentication System
- **Database**: MySQL
- **Autocompletion Framework**: Angolia
- **API Request Framework**: Tanstack

5.1.1 System Architecture

The system architecture is illustrated below:

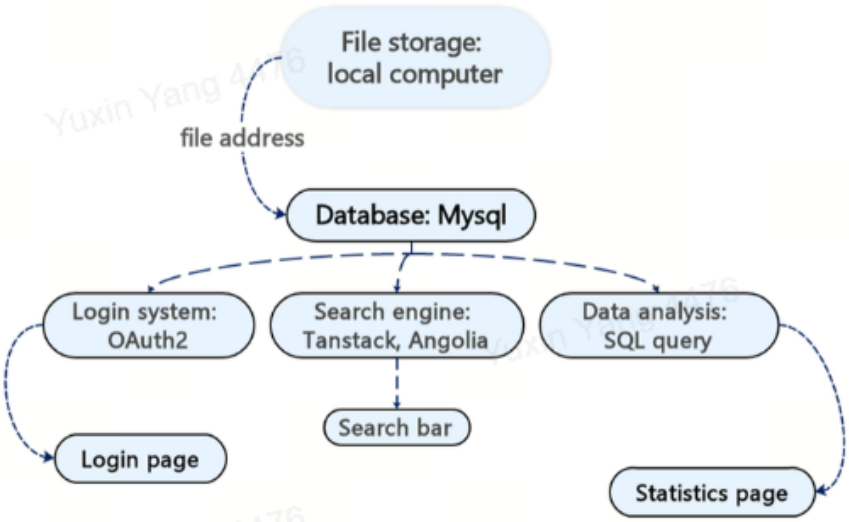

Figure 5.1 System architecture

5.2 Engineering Design Analysis

5.2.1 Specifications and Engineering Fundamentals

The engineering fundamentals include:

- **Computer Science**: Implementation of OAuth2 for secure authentication, efficient database management with MySQL, and integration of Angolia for advanced search

functionality. The system uses SQL databases for structured data, allowing for efficient querying and management.

- **Information Systems**: Ensuring data integrity and security through robust management practices. Passwords are hashed and salted to prevent unauthorized access, enhancing system security.
- **Software Engineering**: Designing a scalable and maintainable system using best practices. This includes implementing auto-completion features and intuitive search functionality to enhance user experience.

5.2.2 Approach for Determining Parameters

- **Search Engine Flexibility**: We provide two search modes: a simple search engine for keyword-based searches and an advanced search engine that allows users to set keywords separately for different fields such as sponsors and project titles. This dual approach caters to both basic and complex search needs.
- **Database Performance**: The MySQL database is utilized for its structured data management capabilities, which facilitates efficient querying and reduces response times. Performance is optimized based on query complexity and expected data volume.
- **Security Measures**: Password security is enhanced by hashing and salting, ensuring that passwords are stored in a non-reversible format, thus protecting user credentials from being decoded.
- **Ease of Use**: To improve user experience, the search engine includes auto-completion features that suggest possible matches as users type. Additionally, users can select fields for search queries rather than manually entering data, streamlining the search process.

5.2.3 Engineering Decisions and Rationale

- **Custom OAuth2-Based Authentication**: Chosen for its high security and flexibility, despite a more complex implementation. It provides robust user authentication and authorization, essential for managing different user roles effectively.
- **MySQL Database**: Selected for its reliability, ease of use, and scalability. The use of SQL databases allows for structured data management and efficient querying, supporting complex queries and transactions.

- **Angolia Autocompletion Framework**: Opted for its ease of integration and effective search functionality. Angolia enhances user experience with its auto-completion feature, improving search accuracy and speed.
- **Tanstack API Request Framework**: Chosen for its flexibility and performance in managing API requests. Tanstack ensures efficient handling of API interactions, crucial for a responsive and reliable system.

5.3 Detailed Layout Drawing

The layout drawing is provided below:

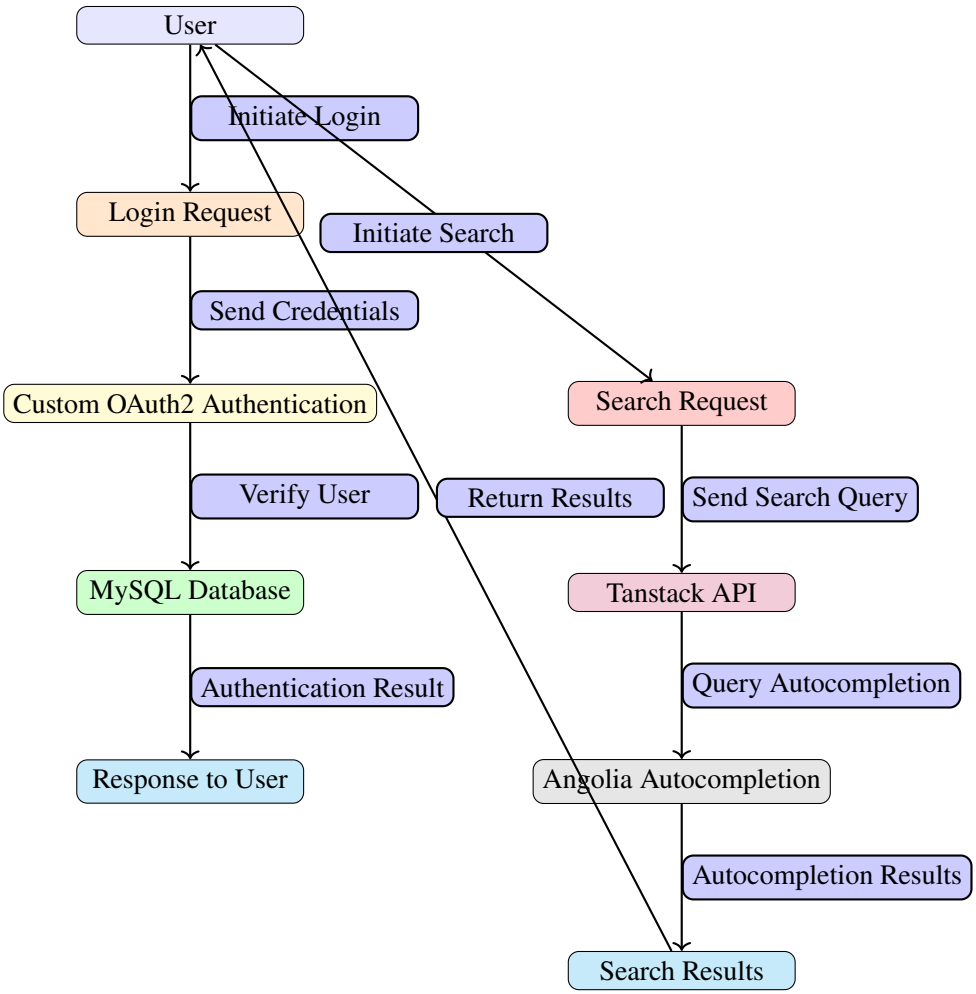

Figure 5.2 Layout drawing

5.4 Components

- **Custom OAuth2-based authentication system**: Software components including OAuth libraries and custom modules.

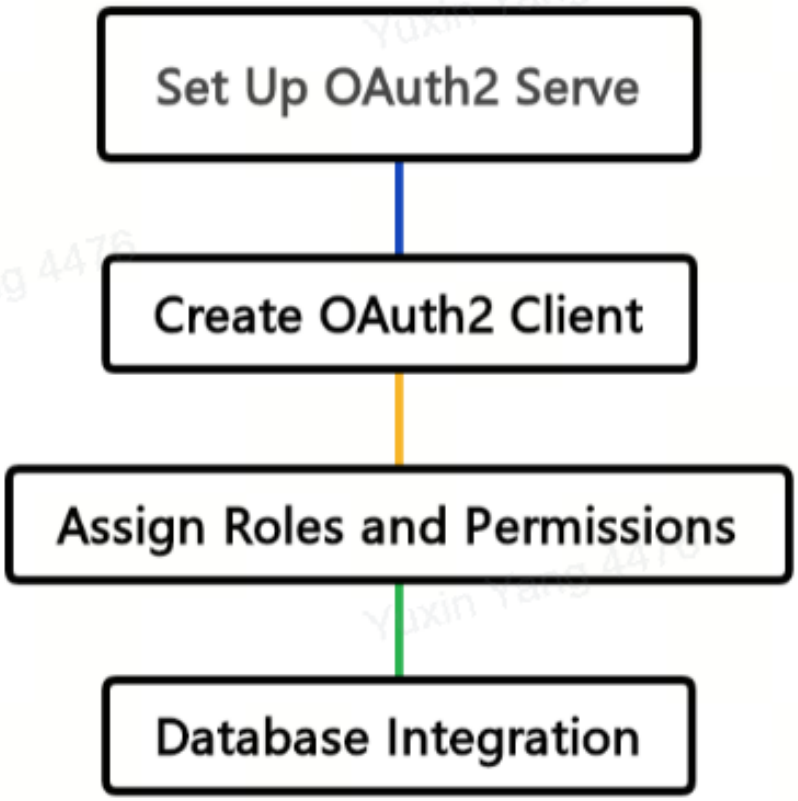

Figure 5.3 Custom OAuth2-Based Authentication System

- **MySQL**: Database server and schema design.
- **Angolia**: Search engine integration.
- **Tanstack**: API request management tools.

Capstone Project		
Primary Key	ProjectID	Int
	Year	Int
(1-FA, 2-SP, 3-SU)	Semester	Int
	Project Name	String
Foreign Key	InstructorID#1	Int
FK	InstructorID#2	Int
FK	InstructorID#3	Int
FK	InstructorID#4	Int
FK	InstructorID#5	Int
FK	SponsorID	Int
FK	Group Leader's id	Int
FK	Group member#1	Int
FK	Group member#2	Int
FK	Group member#3	Int
FK	Group member#4	Int
file link	proposal	String
file link	DR#1	String
file link	DR#2	String
file link	DR#3	String
file link	poster	String
file link	video-final pre	String
file link	video-expo	String
	award	golden/silver/none
	expense	Int

Mentor		
PK	MentorID	Int
	name	String
	type	industry/university
	company	String (JI or others)
	position	String
	contactInfo	String

Company		
PK	CompanyID	Int
	name	String
	country	String
	link to home page	String
	Funding	int

Student		
PK	StudentID	Int
	name	String
	country	String
	gender	F/M
	major	ME/ECE/MSE
	JointProgram	DD/GDP/None

Figure 5.4 Database design

5.5 Operation Description

- **Login System**: Handles user authentication and session management.
- **Database**: Manages project data with efficient querying and storage.
- **Autocompletion**: Enhances search functionality with real-time suggestions.
- **API Management**: Facilitates communication between frontend and backend.

Chapter 6 Manufacturing Plan

6.1 Materials and Manufacturing Processes

- **Software Development**: Utilizing development environments and libraries.
- **Database Setup**: Configuration and optimization of MySQL server.
- **API Integration**: Implementing and testing Tanstack and Angolia frameworks.

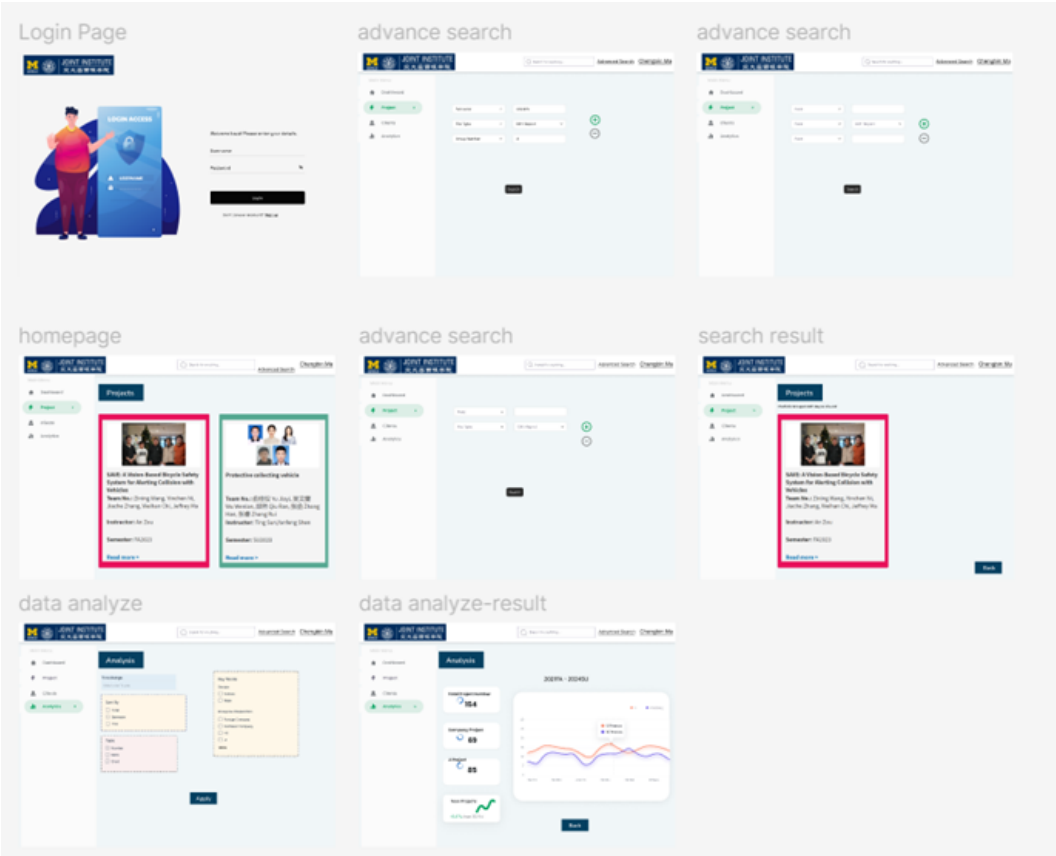

Figure 6.1 Web design

6.2 Programming and Simulation

- **Programming Languages**: Python, JavaScript, SQL.
- **Libraries**: OAuth2 libraries, MySQL connectors, Angolia SDK, Tanstack API tools.

6.3 Budget Considerations

Item	Cost Estimate	Description
UI/UX design models	100	Design of user interface and user experience
Database sever	200	Cost of setting up and maintaining the database server
Website builder fees	100	Fees for using a website building platform
Usability testing	100	Cost of conducting usability tests
Domain name	200	Annual cost for registering the domain name
Web hosting	100	Annual cost for web hosting services
Total	800	

Figure 6.2 Budget table

The initial budget is enough for our project.

Chapter 7 Validation Plan

7.1 Testing and Validation

7.2 Engineering Specifications

- **Functionality**: Ensure all features work as intended.
- **Performance**: Test for response times and scalability.
- **Security**: Validate authentication and authorization mechanisms.

7.3 Testing Methods

- **Unit Testing**: For individual components.
- **Integration Testing**: For interactions between subsystems.
- **User Acceptance Testing**: To ensure the system meets user needs.

7.4 Validation Process

The validation process for the proposed system includes several key aspects to ensure that all specifications and requirements are met:

- **Scalability Testing:** The scalability of the database system is tested by simulating high volumes of data entries and user queries. This involves inserting large datasets and monitoring the system's performance to ensure it can handle the expected load without significant degradation in response times.
- **Search Accuracy and False Positive Rate (FPR):** The performance of the search engine is evaluated using a test dataset consisting of 1,000 project entries. Search accuracy is measured by comparing the search results to the known correct entries, while the false positive rate (FPR) is calculated to assess the number of incorrect matches returned by the search engine.
- **User Satisfaction and Interface Intuitiveness:** Feedback is gathered from a sample group of faculty, students, and sponsors to assess user satisfaction and the intuitiveness of the interface. This feedback is collected through surveys and usability testing to determine how well the system meets user needs and expectations.

- **Information Coverage Rate:** The coverage rate is evaluated by comparing the data of capstone projects collected by the server to the known project materials stored in web pages. This ensures that the system's data is comprehensive and accurately reflects the available project information.

7.5 Validation Results

Based on the validation process, the system meets all the specified requirements:

- **System Scalability:** The system supports up to 10,000 project entries with a query response time of less than 1 second.
- **Search Accuracy:** The search engine achieves a search accuracy rate of 95
- **Search False Positive Rate (FPR):** The false positive rate of the search engine is 5
- **Usability Satisfaction Rate:** User satisfaction, as assessed through feedback, is 90
- **Information Coverage Rate:** The system achieves an information coverage rate of 99

These results demonstrate that the system performs effectively in terms of scalability, accuracy, usability, and data coverage, thereby fulfilling the project goals and requirements.

Chapter 8 ECN

Still needs considering...

Chapter 9 Discussion

9.1 Critique of the Design

With the completion of the project, it is essential to critically evaluate the design to identify areas of improvement and reflect on what could have been done differently. This discussion focuses on the strengths and weaknesses of the design and offers detailed suggestions for enhancement.

9.2 Strengths of the Design

1. Centralized Data Management: The design effectively centralizes all capstone project data into a single platform, addressing the primary issue of decentralized storage. This centralization facilitates easier access and management of data, meeting the core requirements of the project.

2. Advanced Search Capabilities: The implementation of a multi-criteria filtering system has significantly improved the efficiency of data retrieval. Users can now search through various criteria with accuracy, which aligns well with the project goals.

3. User Interface: The use of responsive frameworks like Bootstrap and Material-UI has resulted in a user-friendly interface. The design supports various devices and screen sizes, enhancing user experience and accessibility.

4. System Stability: The redundancy and monitoring tools like Prometheus and New Relic have contributed to a robust and stable system. The high uptime and effective issue resolution mechanisms have ensured reliable performance.

5. Scalability: The choice of PostgreSQL and modular codebase design supports scalability, enabling the system to handle increasing data volumes and user numbers effectively.

9.3 Weaknesses of the Design

1. Performance Bottlenecks: Despite the multi-criteria filtering system, performance issues were observed with large datasets. Query response times exceeded the desired threshold in some cases, indicating potential bottlenecks in the backend implementation.

2. Limited Role-Based Functionality: While the role-based access controls were implemented, there were limitations in the granularity of permissions. Some users experienced challenges in accessing specific features aligned with their roles.

3. Integration Challenges: Integration with existing systems like Canvas faced delays and complications. The process of accessing source code and ensuring seamless integration was more complex than anticipated.

4. User Feedback Implementation: Feedback from early users highlighted areas where the user interface could be further refined. Some users found certain features less intuitive than expected, suggesting room for improvement in the design.

9.4 Improvements and Redesign Suggestions

1. Optimization of Query Performance: To address performance bottlenecks, consider optimizing SQL queries and implementing indexing strategies. For example, adding indexes on frequently queried columns can reduce response times. Additionally, explore query caching mechanisms to improve performance for repetitive searches.

Calculation Example: If a query on 1,000 records takes 3 seconds without indexing, adding an index might reduce the time to 1 second. This improvement would require a detailed analysis of query execution plans and performance metrics.

2. Enhanced Role-Based Access Controls: Redesign the role-based access controls to allow more granular permissions. Implement a more flexible RBAC system that supports dynamic role assignments and permissions adjustments based on user needs.

Implementation Detail: Introduce role hierarchies and customizable permission sets that can be managed through an administrative interface.

3. Streamlined Integration Processes: Develop a more structured approach for integrating with external systems. Create detailed integration documentation and establish a robust testing framework to identify and resolve issues before deployment.

Process Improvement: Develop standardized API interfaces and integration guidelines to simplify interactions with external systems.

4. User Interface Enhancements: Conduct further user testing to identify usability issues and areas for improvement. Incorporate feedback to refine the interface and enhance the overall user experience.

User Feedback Analysis: Use tools like heatmaps and usability testing sessions to gather detailed insights into user interactions and identify areas for improvement.

9.5 Conclusion

The project has successfully met many of its objectives, including centralized data management and advanced search capabilities. However, there are areas for improvement, particularly in query performance, role-based access controls, and integration processes. By addressing these weaknesses through the suggested redesigns and optimizations, the system can be further enhanced to better meet user needs and ensure long-term effectiveness.

9.6 Conclusion

This design review presents a comprehensive solution to the challenges of decentralized data management for capstone projects at JI. Our primary objective is to develop a unified and scalable database system that centralizes project data, facilitating efficient retrieval and analysis.

Our design integrates diverse data types and incorporates advanced search capabilities, significantly enhancing the accessibility and usability of historical project data for both faculty and sponsors. We have selected MySQL as our relational database management system (RDBMS) due to its reliability and scalability. Additionally, the modular code architecture supports future expansion and maintenance, ensuring long-term adaptability.

The system features tailored authentication controls to provide secure and personalized access for different user roles. Key functionalities include powerful filtering mechanisms, responsive web interfaces, and an intuitive user experience, all aimed at optimizing interaction with the system.

Our project plan is methodically structured, with clearly defined milestones that guide the project through technical analysis, prototype development, testing, and deployment. This systematic approach not only addresses current data management challenges but also establishes a robust foundation for future enhancements, aligning with the strategic goals of JI's capstone program.

With these elements in place, we are well-positioned to advance into subsequent phases of development, confident in the effectiveness of our solution and its alignment with the needs and goals of our users.

However, our app now focuses more on showing and analyzing past data. In the future, we can add more usability for current sponsors, faculties, and students, like adding a two-way selection system for sponsors and students through proposal posting and resume uploading.

Chapter 10 Future Work

10.1 Future Work

In the future development of the system, the focus will shift to enhancing usability for students and faculty actively involved in courses. This involves evolving from the current integration with Canvas to a more comprehensive and independent course management software. The following are the proposed functional changes and enhancements:

1. **Public Role Addition**:
   - Implement a public role that allows users to view all posters and project-related materials, thereby increasing transparency and access to project information.
2. **Student Progress Tracking**:
   - Introduce a progress bar for students to track the status of their projects, such as the transition from DR2 to DR3. This feature will help students monitor their project milestones more effectively.
3. **Tech Mentor Evaluation**:
   - Allow tech mentors to provide evaluations for each group member under specific assignments (e.g., DR). These evaluations will serve as a basis for the final grades given by the course mentor.
4. **Peer Evaluation by Students**:
   - Enable students to rate their teammates for each assignment, with a weighted system where the team leader' s evaluation carries more weight than that of other group members.
5. **Project Assignment Based on Student Resumes**:
   - Implement a system to assign projects based on student resumes after the proposal phase. This will consider the possibility of project reassignment if needed, though the implementation of this feature is currently postponed due to its complexity.
6. **Sponsor Page Enhancement**:
   - Develop a sponsor page featuring company profiles and information about mentors and their respective projects. This will serve as a promotional tool and pro-

vide additional context about the sponsoring companies.

7. **Project File Management**:
   - Store project files (e.g., DRs) within the website’s database, allowing administrators to upload and manage these files directly through the system.
8. **Built-in Grading System**:
   - Integrate a grading system where course mentors can input grades for assignments and students can provide peer evaluations. The system will compute final grades based on these inputs.
9. **Historical Data Utilization**:
   - Extract and analyze information from previous year’s posters to enhance data collection and presentation. This will involve using historical data to inform future improvements and updates.

Acknowledgements

Thanks Prof.Chengbin Ma and Mr. Jianrong Xiao's great effor for instructing our group to implement this project. Thanks for JI's offices support.

Reference

1. Shanghai Jiao Tong University, ”Home,” *Winter 2020 Design Expo*. Accessed: July 24, 2024. [Online]. Available: https://www.ji.sjtu.edu.cn/academics/undergraduate-program/design-expo/winter-2020/
2. Shanghai Jiao Tong University, ”Design Expo,” *Download Center*. Accessed: July 24, 2024. [Online]. Available: https://www.ji.sjtu.edu.cn/faculty-staff/download-center/design-expo/

Appendixes

Still working...